第 7 章 无监督学习算法

章节导航

- 了解无监督学习的基本概念。
- 掌握 k 均值聚类、层次聚类和密度聚类等常用聚类算法的核心思想。
- 掌握主成分分析算法的核心思想。
- 能够使用相应算法解决实际问题。

本章知识导图

7.1 聚类算法

聚类算法是机器学习中对样本进行分组所使用的一种算法。针对给定的样本集，通过聚类算法可以将样本集划分为不同的簇，使得簇内样本尽可能相似，同时使簇间样本的差异尽可能大。作为一种常用的数据分析算法，聚类算法在很多领域均有广泛应用。如将图像中不同前景和背景分离的图像分割、根据文本内容的相似性进行网页聚类、通过浏览和购买商品的习惯及行为进行电商用户聚类等，都是应用聚类算法实现的。

为了后文叙述方便，这里重述一下符号：设有 m 个样本，样本集为 $D=\{\boldsymbol{x}_1, \boldsymbol{x}_2, \cdots, \boldsymbol{x}_m\}$，使用聚类算法将样本集 D 划分为 k 个簇，记为 C_l，$l=1, 2, \cdots, k$，其中 $C_l \subset D$。通常情况下，$\{C_1, C_2, \cdots, C_k\}$构成样本集 D 上的一个划分，即满足如下条件：

(1)$C_l \neq \varnothing$；

(2)$D=\bigcup_{l=1}^{k} C_l$；

(3)$C_l \cap C_{l'}=\varnothing$，$l \neq l'$。

条件(2)要求每个样本必须归到某个簇中，条件(3)使得每个样本只能归到一个簇中。

大多数聚类算法所得结果满足上述 3 个条件，但是也有个别算法不满足条件(2)，如密度聚类的代表算法——具有噪声的基于密度的聚类算法(density-based spatial clustering of application with noise，DBSCAN)。使用该算法聚类后，个别样本不属于任何簇，此时这些样本称为噪声样本。

7.1.1 k 均值聚类算法

物以类聚，人以群分。如前文所述，聚类的目标是使得簇内样本尽可能相似，簇间样本的差异尽可能大。换句话说，就是要求簇内的样本非常集中，如果从距离的角度考虑，

就是要求簇内样本到该簇中心的距离尽可能小。当已经确定需要分成 k 个簇时，可以最小化平方误差

$$E=\sum_{i=1}^{k}\sum_{\boldsymbol{x}\in C_i}\|\boldsymbol{x}-\boldsymbol{\mu}_i\|_2^2$$

k 均值聚类案例

其中，$\boldsymbol{\mu}_i=\frac{1}{|C_i|}\sum_{\boldsymbol{x}\in C_i}\boldsymbol{x}$，是第 i 个簇 C_i 的均值，称为聚类中心。

上述优化问题需要考虑集合 D 上所有可能的划分，是一个 NP 难问题，需要使用近似策略求解。k 均值聚类算法使用了贪心策略，从随机选定的聚类中心开始，交替更新簇和聚类中心，直至簇稳定下来。算法描述如下。

(1)从集合 D 中随机选择 k 个样本作为初始的聚类中心：$\boldsymbol{\mu}_1$，$\boldsymbol{\mu}_2$，…，$\boldsymbol{\mu}_k$。

(2)更新簇：依次从集合 D 中选取第 i 个样本 $\boldsymbol{x}_i$，在当前 k 个聚类中心中，找到与 $\boldsymbol{x}_i$ 距离最短的样本，然后将 $\boldsymbol{x}_i$ 归到该聚类中心代表的簇中。

(3)更新聚类中心：计算每个簇的均值，更新聚类中心。

(4)如果所有聚类中心不再发生变化，则停止更新，并输出簇；否则转到第(2)步。

值得注意的是，上述算法的第(2)步可能产生空簇。此时可以利用其他簇的聚类中心重新构建该簇的聚类中心，也可以随机挑选一个样本作为该簇的聚类中心。此外，上述算法中的距离默认是欧几里得距离，也可以使用其他距离。

k 均值聚类算法的原理简单，易于实现，且可解释性强，但结果是局部最优的，且严重依赖初始聚类中心。解决办法是进行多轮聚类，然后选择最优的聚类结果。此外，如果初始聚类中心选择得好的话，可以加快收敛速度，同时也更容易获得全局最优解。因此，如何选择合适的初始聚类中心也得到用户的广泛关注。其中，k 均值＋＋算法基于“聚类中心相互之间分得越开越好”的思想确定初始聚类中心。算法描述如下。

(1)从输入的数据集中随机选择一个样本作为第一个聚类中心 $\boldsymbol{\mu}_1$。

(2)对于数据集中的每一个样本 $\boldsymbol{x}_i$，计算它与已选择的聚类中心中最近聚类中心的距离，即

$$\rho(\boldsymbol{x}_i)=\min_r\|\boldsymbol{x}_i-\boldsymbol{\mu}_r\|_2^2,\ r=1,2,\cdots,k$$

(3)选择一个新的数据点作为新的聚类中心，选择原则是：$\rho(\boldsymbol{x})$较大的点，被选取作为聚类中心的概率较大。实现时按照轮盘法选择，其中，$\boldsymbol{x}$ 被选取的概率为 $\frac{\rho(\boldsymbol{x})^2}{\sum_{\boldsymbol{x}\in D}\rho(\boldsymbol{x})^2}$。

(4)重复步骤(2)和(3)，直到选出 k 个聚类中心。

现在考虑 k 均值聚类算法的运行速度，由算法描述可知，在更新簇时需要计算每个样本与当前聚类中心之间的距离，所以随着样本量的增加，更新簇的计算量也随之增加。同时，在更新聚类中心时，需要计算每个簇的平均值。当样本量增加时，每个临时的簇也有更多的样本，计算平均值就需要更多的时间开销。所以，随着样本量增多，k 均值聚类算法的运行时间也随之增加。在大数据背景下，针对数量庞大的样本，如何在保持聚类效果的同时提升速度，成为众多学者关心的话题。在此背景下，D. 斯卡利(D. Sculley)提出 mini-batch(分批处理)k 均值聚类算法，可以减少 k 均值聚类算法的运行时间，同时其聚类效果只略逊于标准 k 均值聚类算法。

mini-batch k 均值聚类算法的核心思想是：每次随机抽取一个小规模的样本子集来更新聚类中心，而不是像 k 均值聚类算法那样使用所有样本。具体步骤如下。

(1)从集合 D 中随机抽取一个样本子集 M，使用 k 均值聚类算法得到初始的簇和聚类中心。

(2)继续从集合 D 中随机抽取一个样本子集 M，按照距离最小原则，将 M 中的每个样本分配给当前的聚类中心。

(3)基于样本子集 M 更新聚类中心。

(4)如果所有聚类中心都不再发生变化，或者已经达到最大迭代次数，则停止迭代，并输出簇；否则转到第(2)步。

为了考察 mini-batch k 均值聚类算法在大数据方面的优势，这里针对不同数量级的样本分别使用 k 均值聚类算法和 mini-batch k 均值聚类算法进行聚类，记录二者在运行时间、平方误差方面的差异，比较结果如表 7-1 所示。在每次聚类时两种算法均运行 10 次，输出最优结果。mini-batch k 均值算法中每批样本量为 2 048。此外，实验中使用的硬件配置为 Intel E5-1620 型号的中央处理器(central processing unit，CPU)，主频 3.70 GHz，内存 64 GB。

表 7-1　k 均值聚类算法和 mini-batch k 均值聚类算法的比较结果

样本数量	时间/s		平方误差 E		差异率/%
	k 均值聚类算法	mini-batch k 均值聚类算法	k 均值聚类算法	mini-batch k 均值聚类算法	
10^3	0.065 0	0.113 9	1 866.78	1 867.16	0.020 7
10^4	0.084 0	0.102 0	19 233.47	19 237.88	0.022 9
10^5	0.642 7	0.121 9	194 320.43	194 397.52	0.039 7
10^6	4.793 7	0.391 8	1 941 164.31	1 941 929.99	0.039 4
10^7	34.636 3	3.005 6	19 433 152.00	19 450 106.12	0.087 2
10^8	362.478 0	27.765 7	194 223 955.44	194 282 349.42	0.030 1

在数据规模不大时，mini-batch k 均值聚类算法没有优势，运行时间甚至比 k 均值聚类算法还长。但是当数据规模增加到 10^5 以上时，mini-batch k 均值聚类算法在运行速度方面的优势开始显现。比较两种算法的平方误差 E，可以看到 mini-batch k 均值聚类算法均大于 k 均值聚类算法，说明 mini-batch k 均值聚类算法在提升速度时确实牺牲了一些精度。不过从它们的差异率来看，基本稳定在 0.03%左右，不超过 0.1%。特别是随着数据规模增大，二者的差异率没有明显上升。这一事实说明 mini-batch k 均值聚类算法损失的精度不是很大。

此外，分析 k 均值聚类算法和 mini-batch k 均值聚类算法的执行过程可以发现，它们有一个共同点：先初始化聚类中心，然后对其进行迭代并不断更新，直到求出最终的划分结果。这里的聚类中心常称为原型，相应的算法常称为原型聚类，也称基于原型的聚类。这类算法假设聚类结构能够通过一组原型来刻画，在聚类时先初始化原型，然后对原型进行迭代更新求解。

7.1.2　密度聚类算法

原型聚类的代表——k 均值聚类算法直接使用距离进行聚类，与此不同，在聚类结构能通过样本分布的紧密程度确定的假设前提下，密度聚类算法从样本密度的角度来考察样本之间的可连接性，并基于可连接样本不断扩展聚类簇，以获得最终的聚类结果。密度聚类算法也称基于密度的聚类算法，这

密度聚类案例

类算法的代表是 DBSCAN 算法。

为了描述 DBSCAN 算法，这里需要先定义一些基本概念。

对于任意给定样本 $\boldsymbol{x}_i$，$\boldsymbol{x}_i$ 的 ε 邻域定义为与 $\boldsymbol{x}_i$ 距离不超过 ε 的样本集合，记为 $N_\varepsilon(\boldsymbol{x}_i)$，即

$$N_\varepsilon(\boldsymbol{x}_i)=\{\boldsymbol{x}_j\in D \mid \rho(\boldsymbol{x}_i,\ \boldsymbol{x}_j)\leqslant\varepsilon\}$$

其中，$\rho(\cdot,\ \cdot)$表示样本之间的距离。由定义可知，如果 $\boldsymbol{x}_j\in N_\varepsilon(\boldsymbol{x}_i)$，那么 $\boldsymbol{x}_i\in N_\varepsilon(\boldsymbol{x}_j)$。

对于给定的常数 MinPts，如果 $\boldsymbol{x}_i$ 的 ε 邻域中的元素个数不少于 MinPts 个，即 $|N_\varepsilon(\boldsymbol{x}_i)|\geqslant$MinPts，则称 $\boldsymbol{x}_i$ 是一个核心对象。显然，核心对象是指那些周围有较多点的样本，表示它不是孤立地存在于特征空间中。

接下来定义样本间的关系。如果 $\boldsymbol{x}_j\in N_\varepsilon(\boldsymbol{x}_i)$，且 $\boldsymbol{x}_i$ 是核心对象，则称 $\boldsymbol{x}_j$ 由 $\boldsymbol{x}_i$ 密度直达，即核心对象可以密度直达其邻域中的样本。由定义可知，密度直达不具有双向性，即 $\boldsymbol{x}_j$ 由 $\boldsymbol{x}_i$ 密度直达，但是 $\boldsymbol{x}_i$ 由 $\boldsymbol{x}_j$ 密度直达的结论不一定成立。

对于 $\boldsymbol{x}_i$ 和 $\boldsymbol{x}_j$，如果存在样本序列 $\boldsymbol{p}_1$，$\boldsymbol{p}_2$，…，$\boldsymbol{p}_s$，使得 $\boldsymbol{p}_1=\boldsymbol{x}_i$，$\boldsymbol{p}_s=\boldsymbol{x}_j$，且 $\boldsymbol{p}_{i+1}$ 由 $\boldsymbol{p}_i$ 密度直达，则称 $\boldsymbol{x}_j$ 由 $\boldsymbol{x}_i$ 密度可达。由密度直达的定义可知，这里 $\boldsymbol{p}_1$，$\boldsymbol{p}_2$，…，$\boldsymbol{p}_{s-1}$ 均是核心对象。与密度直达一样，密度可达也不具有双向性，即 $\boldsymbol{x}_j$ 由 $\boldsymbol{x}_i$ 密度可达，但是 $\boldsymbol{x}_i$ 由 $\boldsymbol{x}_j$ 密度可达不一定成立。

对于 $\boldsymbol{x}_i$ 和 $\boldsymbol{x}_j$，如果存在 $\boldsymbol{x}_k$，使得 $\boldsymbol{x}_i$ 和 $\boldsymbol{x}_j$ 均由 $\boldsymbol{x}_k$ 密度可达，则称 $\boldsymbol{x}_i$ 与 $\boldsymbol{x}_j$ 密度相连。显然，密度相连具有双向性，即如果 $\boldsymbol{x}_i$ 与 $\boldsymbol{x}_j$ 密度相连，那么 $\boldsymbol{x}_j$ 与 $\boldsymbol{x}_i$ 必然密度相连。

有了密度相连和密度可达的概念之后，在给定邻域参数 ε 和 MinPts 的情况下，便可以定义基于密度的簇。设集合 C 是集合 D 的非空子集，如果集合 C 满足如下条件，那么就称集合 C 是一个簇。

(1)连接性：集合 C 中任意两个样本均密度相连，如果 $\boldsymbol{x}_i\in C$，且 $\boldsymbol{x}_j\in C$，那么 $\boldsymbol{x}_i$ 与 $\boldsymbol{x}_j$ 密度相连。

(2)最大性：由集合 C 中样本密度可达的样本都属于集合 C，如果 $\boldsymbol{x}_i\in C$，且 $\boldsymbol{x}_j$ 由 $\boldsymbol{x}_i$ 密度可达，那么 $\boldsymbol{x}_j\in C$。

也就是说，簇是由密度可达关系导出的最大密度相连的样本集合。

按照以上关于簇的定义，只要从核心对象出发，寻找所有能够密度可达的样本，即可得到一个簇。因此 DBSCAN 算法包含两个主要步骤：首先找到所有核心对象，其次从每个核心对象出发寻找所有该核心对象能够密度可达的样本集合。在寻找过程中，已经被归到某一簇的核心对象将不再被考虑。算法的详细描述如下。

输入：样本集 $D=\{\boldsymbol{x}_1,\ \boldsymbol{x}_2,\ \cdots,\ \boldsymbol{x}_m\}$，邻域参数 ε 和 MinPts，样本间距离的度量方式 $\rho(\cdot,\ \cdot)$。

输出：簇划分 $C=\{C_1,\ C_2,\ \cdots,\ C_k\}$。

算法的步骤描述如下。

(1)初始化核心对象集合 $\Omega=\varnothing$，初始化聚类簇数 $k=0$，初始化未访问样本集合 $\Gamma=D$，簇划分 $C=\varnothing$。

(2)对于 $i=1,\ 2,\ \cdots,\ m$，按下面的步骤找出所有的核心对象。

①通过距离度量方式，找到 $\boldsymbol{x}_i$ 的 ε 邻域 $N_\varepsilon(\boldsymbol{x}_i)$。

②如果样本子集的样本个数满足 $|N_\varepsilon(\boldsymbol{x}_i)| \geqslant \text{MinPts}$，则将样本 $\boldsymbol{x}_i$ 加入核心对象样本集合，$\Omega=\Omega\cup\{\boldsymbol{x}_i\}$。

(3)如果核心对象集合 $\Omega=\varnothing$，则算法结束，否则继续。

(4)在核心对象集合 Ω 中，随机选择一个核心对象 $\boldsymbol{o}$，初始化当前簇的核心对象队列 $Q=\{\boldsymbol{o}\}$，初始化类别序号 $k=k+1$，初始化当前簇 $C_k=\{\boldsymbol{o}\}$，更新未访问样本集合 $\Gamma=\Gamma-\{\boldsymbol{o}\}$。

(5)更新核心对象集合 $\Omega=\Omega-C_k$，如果 $Q=\varnothing$，则当前簇 C_k 生成完毕，更新划分 $C=C\cup\{C_k\}$，转到步骤(3)；否则继续。

(6)在当前簇的核心对象队列 Q 中取出一个核心对象 $\boldsymbol{o}'$，找到 $\boldsymbol{o}'$ 的 ε 邻域 $N_\varepsilon(\boldsymbol{o}')$，令 $\Delta=N_\varepsilon(\boldsymbol{o}')\cap\Gamma$，更新当前簇 $C_k=C_k\cup\Delta$，更新未访问样本集合 $\Gamma=\Gamma-\Delta$，更新 $Q=Q\cup(\Delta\cap\Omega)-\{\boldsymbol{o}'\}$，转入步骤(5)。

相比于 k 均值聚类算法，DBSCAN 算法无须指定簇的个数，这在很大程度上带来了方便。但是需要指定邻域参数 ε 和 MinPts，这两个参数对结果的影响很大，实际应用时需要联合调参。

使用 DBSCAN 算法，可能产生一些异常点，这些点最终没有被归到任何簇中，这些点称为噪声点。这也是 DBSCAN 算法的一个优势，可以有效避免少量游离于簇外的噪声点的影响。噪声点的显著特点是，它们不在任何一个核心对象的周围。这一特性破坏了划分的条件：$D=\bigcup_{l=1}^{k}C_l$。

在定义邻域时，用到了距离的度量 $\rho(\cdot,\cdot)$。一般情况下使用欧几里得距离，也可以使用其他距离。如果样本量较大，也可以考虑使用 kd 树、球树等数据结构来加速邻域的搜索。

如前文所述，与 k 均值聚类相关的算法受初始值的影响较大。相对来说，DBSCAN 算法较为稳定，但是它也不是完全稳定，结果有时与步骤(4)中被选核心对象的顺序有关系。事实上，如果某个样本到两个核心对象的距离都小于 ε，即该样本同在两个核心对象的 ε 邻域中，而这两个核心对象由于不是密度直达又不属于同一个簇，从簇的定义来看，此时该样本同属于这两个簇。从 DBSCAN 算法的描述可以看到，它采用了先来后到的规则，这个样本最终会被归到先构建的簇中。

因此，基于密度可达和密度相连定义的簇也不满足划分的另一个条件：$C_l\cap C_{l'}=\varnothing$，$l\neq l'$。但是，使用 DBSCAN 算法得到的簇是满足该条件的。

7.1.3　层次聚类算法

与原型聚类算法和密度聚类算法不同，层次聚类算法不再产生单一的聚类结果，而是产生一个聚类层次，在不同层次对数据集进行划分，从而形成树形的聚类结构。数据集的划分可采用自顶向下的拆分策略，也可采用自底向上的聚合策略，相应的算法分别称为分裂式层次聚类算法和凝聚式层次聚类算法。

聚类算法对比

分裂式层次聚类算法在初始阶段将所有样本的集合视为一个簇，之后按照某种既定的规则(如最大欧几里得距离)逐步分裂出越来越小的簇，直到每个样本自成一簇，或者达到某个终止条件(如簇数目或者簇距离达到阈值)。整个过程自顶向下进行，将簇逐渐拆分，最终也会形成一棵树。该类算法的代表是 DIANA(divisive analysis，分裂分析)算法。

与分裂式层次聚类算法恰好相反，凝聚式层次聚类算法在初始阶段将每个样本都视为一个簇，之后按照某种规则合并最接近的两个簇，直到最后只剩一个簇，或者达到某个终

止条件。整个过程自底向上进行，将所有样本逐渐合并，最终形成一棵树。该类算法的代表是 AGNES(agglomerative nesting，凝聚嵌套)算法。

这里以 AGNES 算法为例给出其步骤描述。

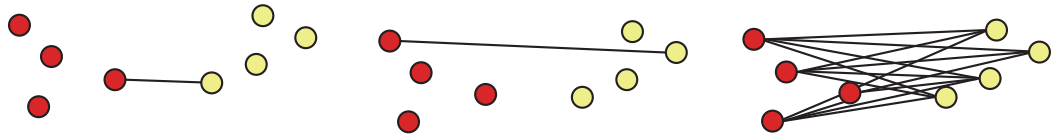
(1)初始化，将每个样本都视为一个簇。

(2)计算各簇之间的距离。

(3)寻找最近的两个簇，并将它们合并为一个簇。

(4)重复步骤(2)、(3)，直到所有样本归为一个簇。

上述算法需要计算各簇之间的距离。事实上，不管层次聚类算法是分裂式还是凝聚式，通常都会涉及簇之间的相似度。最常用的方式就是用簇之间的距离来描述相似度，距离越大，相似度越小。那么如何度量簇之间的距离？考虑到每个簇是一个样本集合，所以只需考虑集合之间的距离即可。这里列举常用的 3 种度量集合之间距离的方法，分别是最小距离、最大距离和平均距离，如图 7-1 所示。



（a）最小距离　　（b）最大距离　　（c）平均距离

图 7-1　簇间距离示意图

设 C_i 和 C_j 是两个簇，$\rho(\boldsymbol{x}, \boldsymbol{z})$是样本之间的距离，它们之间的最小距离定义为

$$d_{\min}(C_i, C_j) = \min_{\boldsymbol{x} \in C_i, \boldsymbol{z} \in C_j} \rho(\boldsymbol{x}, \boldsymbol{z})$$

即簇间最近的两个样本之间的距离为簇间最小距离。最大距离定义为

$$d_{\max}(C_i, C_j) = \max_{\boldsymbol{x} \in C_i, \boldsymbol{z} \in C_j} \rho(\boldsymbol{x}, \boldsymbol{z})$$

即簇间最远的两个样本之间的距离为簇间最大距离。平均距离定义为

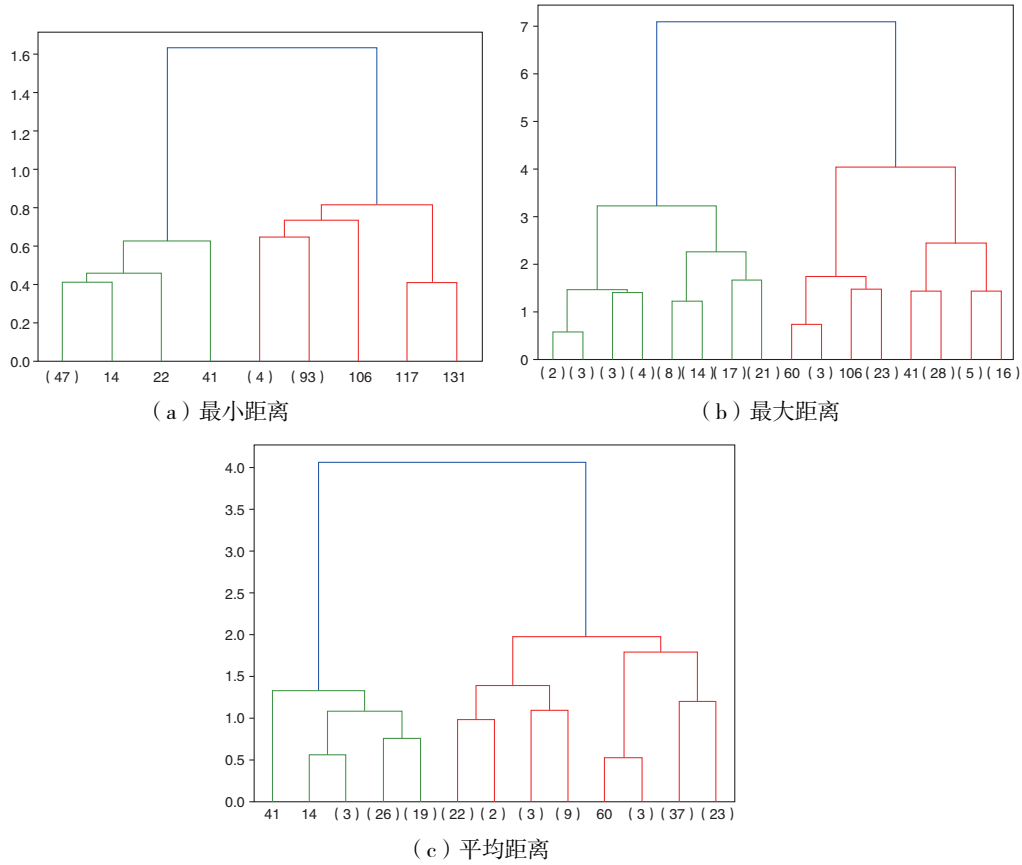
$$d_{\text{avg}}(C_i, C_j) = \frac{1}{|C_i||C_j|} \sum_{\boldsymbol{x} \in C_i} \sum_{\boldsymbol{z} \in C_j} \rho(\boldsymbol{x}, \boldsymbol{z})$$

即簇间所有样本的平均距离为簇间平均距离。

为了直观感受层次聚类算法的树形结果，这里针对鸢尾花数据集(共 150 个样本，分为 3 个类别，每个类别有 50 个样本)，分别选用上述最小距离、最大距离和平均距离，使用 AGNES 算法进行聚类，结果如图 7-2 所示。其中，横坐标中括号内的数字表示当前节点中的样本个数，没有括号的数字表示样本序号；纵坐标是对应的距离值。

图 7-2(a)中的阈值如果取为 0.79，则可以划分为 3 个类别，这 3 个类别的样本个数分别为 50、98、2；图 7-2(b)中的阈值如果取为 3.5，则可以划分为 3 个类别，这 3 个类别的样本个数分别为 72、28、50；图 7-2(c)中的阈值如果取为接近 1.9，也可以划分为 3 个类别，这 3 个类别的样本个数分别为 50、36、64。结合鸢尾花数据集有 3 个类别、每个类别有 50 个样本的特点，可以发现采用最小距离的聚类结果最差(最不均衡)，基于平均距离的聚类结果最优(最均衡)。这个发现具有一般性，大多数情况下，与最小距离和最大距离相比，采用平均距离总是能够得到更理想的聚类结果。此外，基于不同距离的聚类结果中，

都有一类的样本个数为 50 簇，这与鸢尾花数据集的特点是吻合的，即数据集中有一类和另外两类是完全线性可分的。



（a）最小距离　　（b）最大距离

（c）平均距离

图 7-2　使用 AGNES 算法对鸢尾花数据集聚类的结果

7.1.4　聚类模型的评价

与分类模型的评价一样，如何评价聚类模型也是一个重要的问题。评价指标通常有两种：外部指标和内部指标。如果通过参考模型或者其他手段能够获得聚类参考结果，那么可以通过与该聚类参考结果进行比较，从而获得评价指标，这种指标称为外部指标。反之，根据簇内相似度高且簇间相似度低的要求，直接利用聚类结果的指标来评价，而不利用任何参考模型给出的评价指标，这种指标称为内部指标。

1. 外部指标

首先考虑有参考结果的情况，这里主要介绍两个常用的外部指标：兰德指数和调整的兰德指数。当参考结果已知时，评价模型最朴素的想法就是计算正确聚类的占比情况。为了叙述方便，引入如下记号。

(1)TP：表示在参考聚类结果中属于同一簇的两个样本，模型预测也在同一簇的情况的数量。

(2)FP：表示在参考聚类结果中不属于同一簇的两个样本，模型预测却在同一簇的情况

的数量。

(3)TN：表示在参考聚类结果中不属于同一簇的两个样本，模型预测也不在同一簇的情况的数量。

(4)FN：表示在参考聚类结果中属于同一簇的两个样本，模型预测却不在同一簇的情况的数量。

显然，模型预测正确聚类的数量是 TP＋TN，样本两两一组的数量有 TP＋FP＋TN＋FN，假设样本数量为 m，不难验证 $\mathrm{TP}+\mathrm{FP}+\mathrm{TN}+\mathrm{FN}=\frac{m(m-1)}{2}$。因此，模型预测正确的比例是

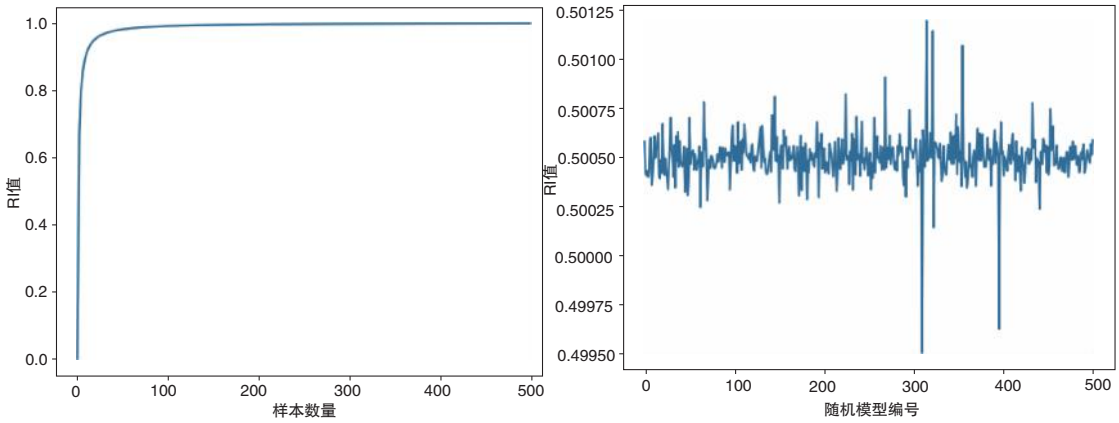
$$\frac{\mathrm{TP}+\mathrm{TN}}{\mathrm{TP}+\mathrm{FP}+\mathrm{TN}+\mathrm{FN}}=\frac{2(\mathrm{TP}+\mathrm{TN})}{m(m-1)}$$

该指标称为兰德指数(Rand index，RI)。

显然，兰德指数的取值范围是[0，1]，聚类效果越好，该指标越接近 1，指标越接近 0，表明聚类效果越差。但是反过来却不一定成立。有时指标接近 1，聚类效果却很差；有时效果不理想，指标却不接近 0。为了佐证这一结论，这里设计两个实验。

实验 1：现有 N 个簇，每个簇中有 2 个样本，则共有 $2N$ 个样本。假设模型将每个样本单独聚为一类，即模型预测有 $2N$ 个簇。当 N 取 1，2，…，250 时，分别计算模型的兰德指数，结果如图 7-3(a)所示，其中，横坐标是样本数量，纵坐标是兰德指数。注意到当 N 不断增加时，模型的聚类结果与参考结果越来越不同，说明模型的效果越来越差。但是从结果看，兰德指数却在不断增加，并且越来越趋于 1。这说明兰德指数并不总是可靠的，这一情况通常发生在 TN 比 TP 大很多的时候。

实验 2：现有 1 000 个样本，平均分为 2 类。假设模型是随机进行聚类，即随机确定簇的个数，然后随机确定每个样本的归属。共产生 500 个随机聚类模型，它们的兰德指数如图 7-3(b)所示。从结果来看，随机模型的兰德指数基本在 0.5 左右，这与人们的固有认知不符。随机模型的效果本应很差，指标为 0 更符合人们的认知。



（a）实验1：指标接近1，聚类效果却很差　　（b）实验2：随机模型的指标不接近0

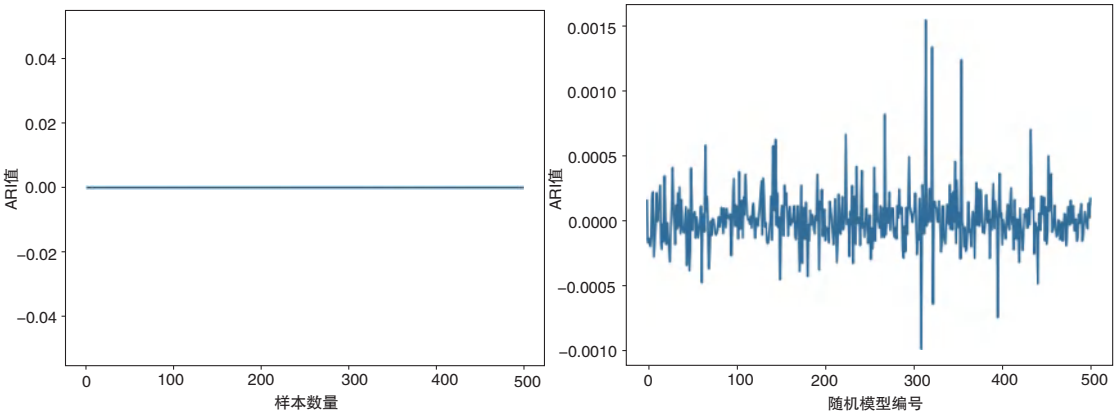
图 7-3　兰德指数存在的问题

调整的兰德指数(adjusted rand index，ARI)可以解决以上问题。设参考聚类结果为 $C^*=\{C_1^*, C_2^*, \cdots, C_s^*\}$，模型聚类结果为 $C=\{C_1, C_2, \cdots, C_t\}$，令 $m_{ij}=|C_i^* \cap C_j|$，

$a_i = \sum_j m_{ij}$，$b_j = \sum_i m_{ij}$，则调整的兰德指数定义为

$$\mathrm{ARI} = \frac{\sum_i \sum_j C_{m_{ij}}^2 - \frac{\sum_i C_{a_i}^2 \sum_j C_{b_j}^2}{C_m^2}}{\frac{1}{2}\left(\sum_i C_{a_i}^2 + \sum_j C_{b_j}^2\right) - \frac{\sum_i C_{a_i}^2 \sum_j C_{b_j}^2}{C_m^2}}$$

针对上述实验 1 和 2，使用调整的兰德指数所得结果如图 7-4 所示。与图 7-3 对比来看，调整的兰德指数很好地避免了效果差但指标却接近 1 的问题。同时，随机模型的指标基本接近 0，这比兰德指数更符合人们的认知。



（a）实验1：指标均为0　　（b）实验2：随机模型的指标接近0

图 7-4　使用调整的兰德指数的聚类结果

此外，调整的兰德指数的取值范围是[−1，1]，值越大，聚类效果越好，取负值的时候，聚类效果很糟，甚至比随机聚类模型的效果还差。

2. 内部指标

内部指标通常根据簇内相似度高且簇间相似度低的要求直接定义评价指标。相似度可以通过距离来描述，距离越大，相似度越低，所以只要考虑簇内样本距离和簇间样本距离即可。

首先考虑单个样本的情况。设样本集 $D = \{\boldsymbol{x}_1, \boldsymbol{x}_2, \cdots, \boldsymbol{x}_m\}$，针对样本 $\boldsymbol{x}_i$，用 a_i 表示该样本到所在簇中其他样本的平均距离，该值体现了簇内相似度，值越小，相似度越大，也意味着凝聚度越大；用 b_i 表示该样本到其他簇的最小距离，该值体现了簇间相似性，值越大，相似度越小，也意味着差异性越大，如图 7-5 所示。这里样本到簇的距离是指样本到该簇内所有样本的平均距离。

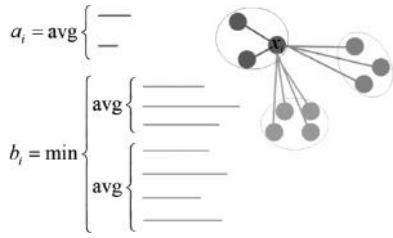


图 7-5　簇内凝聚度和簇间差异性的度量

基于 a_i 和 b_i，定义该样本的轮廓系数

$$s_i = \frac{b_i - a_i}{\max\{a_i, b_i\}}$$

显然，$-1 \leqslant s_i \leqslant 1$。这里有一种特殊情况需要考虑，如果 $\boldsymbol{x}_i$ 所在簇只有这一个样本，

那么 a_i 将无法定义，此时将 s_i 定义为 0，即区间[－1，1]的中点。

以上是单个样本的轮廓系数，聚类结果的轮廓系数是所有样本轮廓系数的平均值，即

$$S=\frac{1}{m}\sum_{i=1}^{m}s_i$$

显然，轮廓系数 S 的取值范围也是[－1，1]，该值越大表明聚类效果越好，该值越小表明聚类结果越差。

回顾 7.1.1 节中的 k 均值聚类算法，所提目标是平方误差 $E=\sum_{i=1}^{k}\sum_{\boldsymbol{x}\in C_i}\|\boldsymbol{x}-\boldsymbol{\mu}_i\|_2^2$ 达到最小，这里 $\boldsymbol{\mu}_i=\frac{1}{|C_i|}\sum_{\boldsymbol{x}\in C_i}\boldsymbol{x}$ 是第 i 个簇 C_i 的均值。因此，平方误差 E 也可以用来评价聚类模型。在簇的个数相同的情况下，该值越小，聚类效果越好。

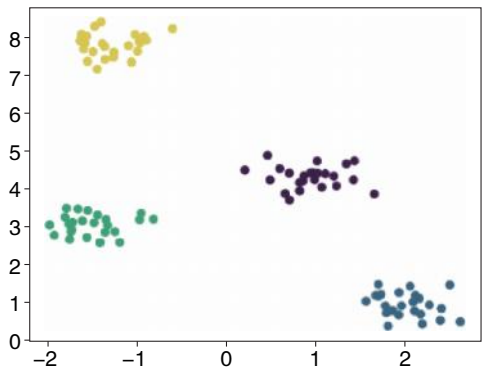
3. 确定簇数

回顾前文，k 均值聚类算法通过显式地指定 k 来决定簇的个数；层次聚类算法需要人工设定阈值从而确定簇的个数；密度聚类算法尽管不需要指定簇的个数，但是需要预先确定邻域参数 ε 和 MinPts，而这两个参数会直接决定簇的个数。因此，如何确定簇的个数显得尤为重要。

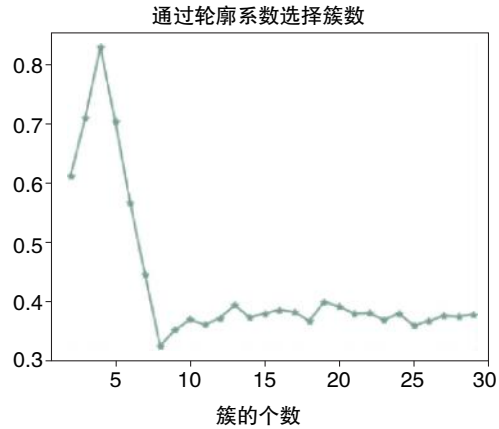
通常可以借助内部指标来选择合适的簇的个数。通用的做法是考察 k 的变化对内部指标的影响，根据变化情况确定最优的 k。这里以 k 均值聚类算法为例给出分析，其他聚类算法可以类似考虑。

如前文所述，轮廓系数大的聚类效果好。因此，可以针对不同的 k 进行聚类，计算对应的轮廓系数，然后选择轮廓系数最大的 k。如图 7-6(b)所示，针对图 7-6(a)中的数据，当 $k=4$ 时轮廓系数最大，那么选择聚类为 4 个簇显然是合理的。

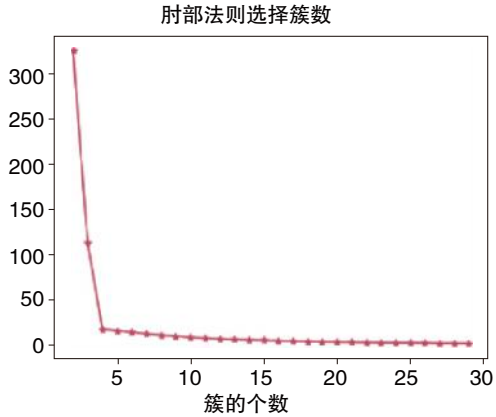
如果使用平方误差，则不能依据指标最小来确定簇数。如图 7-6(c)所示，随着簇数增加，平方误差会逐渐减小，使用平方误差最小作为依据显然不合理。仔细观察，在 k 从 2 变化到 4 的过程中，平方误差急速减小，但是当 k 大于 4 时，降幅突然趋缓。这个降幅突然趋缓的转折点常称为肘点，其因形似手臂的肘部而得名。这就得到了选择簇数的肘部法则：画出平方误差随簇数变化的曲线图，找到肘点并将其作为簇数。



（a）数据



（b）轮廓系数曲线图



（c）平方误差曲线图

图 7-6　*k* 均值聚类算法在不同簇数下的内部指标曲线图

7.2 降维问题

在实际问题中，有时特征的维度很高。一方面，维度高带来较高的运算成本；另一方面，高维度的特征容易出现数据冗余。这一现象正是机器学习中经常会碰到的维度灾难问题，即在高维数据情形下会出现数据样本稀疏、距离计算困难等问题，而且在高维数据中特征之间容易具有线性相关性，这意味着特征出现冗余。

在此背景下，降维算法应运而生：维度灾难为降维提供了需求，特征冗余为降维提供了可能。此外，降维有助于直观地理解特征，因为理解几百个维度的数据结构很困难，但是两三个维度的数据结构可以通过可视化更直观地理解。

7.2.1　主成分分析

降维的目标是在信息损失最小的前提下，尽可能地压缩数据，也就是压缩数据的同时要让信息量达到最大。为了理解降维的思想，先来考察五位学生两个学期的成绩，成绩表如表 7-2 所示。

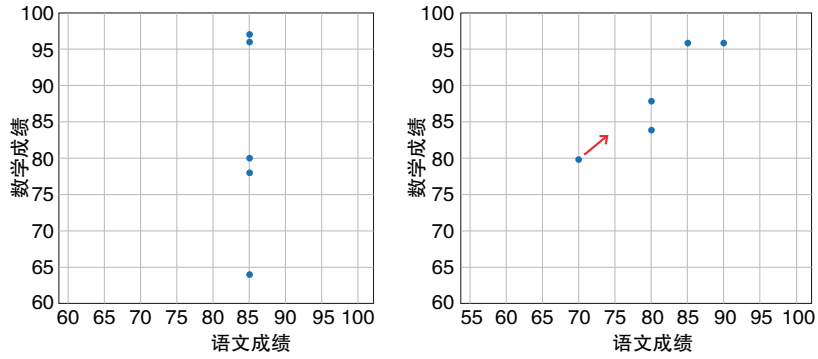
表 7-2　五位学生两个学期的成绩表

学期	科目	学生 1	学生 2	学生 3	学生 4	学生 5
第一学期	语文	85	85	85	85	85
	数学	64	78	80	96	97
第二学期	语文	70	80	80	85	90
	数学	80	84	88	96	96

先看第一学期的情况，可以发现这五位学生的语文成绩相同，没有任何差别，所以此时只需要使用数学成绩即可区分不同学生。第一学期成绩散点图如图 7-7(a)所示，其中横坐标为语文成绩，纵坐标是数学成绩。从该散点图可以看出，只需将二维数据(语文、数学

成绩)投影到纵轴(数学成绩轴)即可。

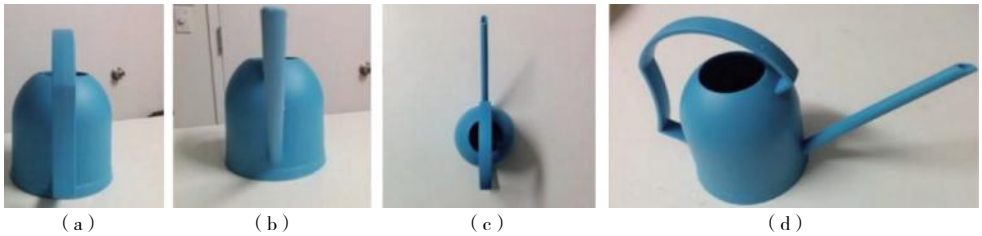
接下来看第二学期的情况，其散点图如图 7-7(b)所示。从散点图来看，横轴和纵轴显然都不是理想的投影方向：将数据投影到横轴(只考虑语文成绩)，无法区分学生 2 和学生 3；而将数据投影到纵轴(只考虑数学成绩)，又无法区分学生 4 和学生 5。从直观来看，此时将数据投影到箭头所指方向，应该能够最大限度地区分不同学生。从统计角度来看，投影以后方差越大的方向，样本在该方向上的区分度越大，意味着该方向包含的信息量越大。通常将投影后方差最大的方向称为第一主成分。



（a）第一学期成绩　　（b）第二学期成绩

图 7-7　五位学生两个学期成绩散点图

以上问题中，只要投影到第一主成分即可，即通过保留第一主成分的信息实现了降维。但是有时可能需要保留更多主成分。比如，在对三维模型进行观察时，实际上是将其投影到二维平面，如图 7-8 所示。不同投影面上的信息量显然有差异，前三种投影的信息量显然不足：(a)缺少壶嘴信息，(b)缺少壶把信息，(c)缺少壶身信息。这三种投影显然不是最理想的降维方式，反观(d)，壶的重要信息(第一和第二主成分)均保留下来，这是理想的降维方式。



（a）　（b）　（c）　（d）

图 7-8　三维特征的降维示例

因此，如何确定主成分便成为降维的核心问题。基于这种思想的降维方法称为主成分分析(principal component analysis，PCA)，该算法是众多降维方法中的一种，属于线性降维方法。

先来考虑如何确定第一主成分，为此考察样本在固定方向上投影后的方差情况。与前文记号一致，设样本集为 $D=\{\boldsymbol{x}_1, \boldsymbol{x}_2, \cdots, \boldsymbol{x}_m\}$，样本均值为 $\boldsymbol{\mu}=\frac{1}{m}\sum_{i=1}^{m}\boldsymbol{x}_i$。用 $\boldsymbol{u}_1$ 表示投影方向，因为这里只考虑方向，并不关心其长度，所以不失一般性，假设 $\boldsymbol{u}_1$ 是单位向量，即

$\|\boldsymbol{u}_1\|_2=1$。向量 $\boldsymbol{x}_i$ 在 $\boldsymbol{u}_1$ 上的投影可以表示为它们的内积，即 $\boldsymbol{x}_i\cdot\boldsymbol{u}_1=\boldsymbol{x}_i^{\mathrm{T}}\boldsymbol{u}_1$。注意到 $\frac{1}{m}\sum_{i=1}^{m}\boldsymbol{x}_i\cdot\boldsymbol{u}_1=\left(\frac{1}{m}\sum_{i=1}^{m}\boldsymbol{x}_i\right)\cdot\boldsymbol{u}_1=\boldsymbol{\mu}\cdot\boldsymbol{u}_1$，即投影的均值等于均值的投影，所以投影后的方差为

$$
\begin{aligned}
&\frac{1}{m}\sum_{i=1}^{m}(\boldsymbol{x}_i\cdot\boldsymbol{u}_1-\boldsymbol{\mu}\cdot\boldsymbol{u}_1)^2\\
&=\frac{1}{m}\sum_{i=1}^{m}[(\boldsymbol{x}_i-\boldsymbol{\mu})\cdot\boldsymbol{u}_1]^2\\
&=\frac{1}{m}\sum_{i=1}^{m}[(\boldsymbol{x}_i-\boldsymbol{\mu})\cdot\boldsymbol{u}_1][(\boldsymbol{x}_i-\boldsymbol{\mu})\cdot\boldsymbol{u}_1]\\
&=\frac{1}{m}\sum_{i=1}^{m}\boldsymbol{u}_1^{\mathrm{T}}(\boldsymbol{x}_i-\boldsymbol{\mu})\cdot(\boldsymbol{x}_i-\boldsymbol{\mu})^{\mathrm{T}}\cdot\boldsymbol{u}_1\\
&=\frac{1}{m}\boldsymbol{u}_1^{\mathrm{T}}\left(\sum_{i=1}^{m}(\boldsymbol{x}_i-\boldsymbol{\mu})(\boldsymbol{x}_i-\boldsymbol{\mu})^{\mathrm{T}}\right)\boldsymbol{u}_1
\end{aligned}
$$

这里 $\frac{1}{m}\sum_{i=1}^{m}(\boldsymbol{x}_i-\boldsymbol{\mu})(\boldsymbol{x}_i-\boldsymbol{\mu})^{\mathrm{T}}$ 是样本的协方差矩阵。令 $\boldsymbol{C}=\frac{1}{m}\sum_{i=1}^{m}(\boldsymbol{x}_i-\boldsymbol{\mu})(\boldsymbol{x}_i-\boldsymbol{\mu})^{\mathrm{T}}$，上述方差表示为 $\boldsymbol{u}_1^{\mathrm{T}}\boldsymbol{C}\boldsymbol{u}_1$。因此，第一主成分便是使得投影后方差 $\boldsymbol{u}_1^{\mathrm{T}}\boldsymbol{C}\boldsymbol{u}_1$ 最大的方向，且该方向是单位向量，即 $\|\boldsymbol{u}_1\|_2=1$。

注意到 $\|\boldsymbol{u}_1\|_2=\boldsymbol{u}_1^{\mathrm{T}}\boldsymbol{u}$，可得如下优化问题：

$$\boldsymbol{u}_1=\arg\max\boldsymbol{u}^{\mathrm{T}}\boldsymbol{C}\boldsymbol{u},\ \boldsymbol{u}^{\mathrm{T}}\boldsymbol{u}=1$$

构造拉格朗日函数

$$L=\boldsymbol{u}^{\mathrm{T}}\boldsymbol{C}\boldsymbol{u}+\lambda(1-\boldsymbol{u}^{\mathrm{T}}\boldsymbol{u})$$

令 $\frac{\partial L}{\partial\boldsymbol{u}}=0$，可得 $2\boldsymbol{C}\boldsymbol{u}-2\lambda\boldsymbol{u}=0$，从而得到所求第一主成分应该满足 $\boldsymbol{C}\boldsymbol{u}=\lambda\boldsymbol{u}$。将 $\boldsymbol{C}\boldsymbol{u}=\lambda\boldsymbol{u}$ 代入目标函数，得到在第一主成分上投影的方差为

$$\boldsymbol{u}^{\mathrm{T}}\boldsymbol{C}\boldsymbol{u}=\boldsymbol{u}^{\mathrm{T}}\lambda\boldsymbol{u}=\lambda\boldsymbol{u}^{\mathrm{T}}\boldsymbol{u}=\lambda$$

注意到满足 $\boldsymbol{C}\boldsymbol{u}=\lambda\boldsymbol{u}$ 的 λ 正是 $\boldsymbol{C}$ 的特征根，而 $\boldsymbol{u}$ 正是属于 λ 的单位特征向量，所以样本在 $\boldsymbol{u}$ 上投影的方差正好是该特征根 λ。因此只要取 $\boldsymbol{C}$ 的最大特征根，便可得到最大方差。为此，只要求出 $\boldsymbol{C}$ 的最大特征根，那么属于该特征根的特征向量便是第一主成分。以此类推，将特征根从大到小排序，分别求出属于这些特征根的特征向量，便可依次得到第二主成分、第三主成分等。

基于上述讨论，可以得到如下算法。

输入：样本集 $D=\{\boldsymbol{x}_1,\boldsymbol{x}_2,\cdots,\boldsymbol{x}_m\}$；降维后的维度 d。

步骤：

(1)计算样本均值 $\boldsymbol{\mu}=\frac{1}{m}\sum_{i=1}^{m}\boldsymbol{x}_i$；

(2)中心化样本 $\boldsymbol{x}_i\leftarrow\boldsymbol{x}_i-\boldsymbol{\mu}$；

(3)计算 $\boldsymbol{C}=\boldsymbol{X}\boldsymbol{X}^{\mathrm{T}}$，其中 $\boldsymbol{X}=[\boldsymbol{x}_1,\boldsymbol{x}_2,\cdots,\boldsymbol{x}_m]_{n\times m}$；

(4)对 $\boldsymbol{C}$ 做特征根分解；

(5)取最大的 d 个特征根对应的特征向量 $\boldsymbol{u}_1$，$\boldsymbol{u}_2$，…，$\boldsymbol{u}_d$。

输出：投影矩阵 $\boldsymbol{W}=[\boldsymbol{u}_1, \boldsymbol{u}_2, \cdots, \boldsymbol{u}_d]_{n\times d}$。

降维过程：$\boldsymbol{x}_{n\times 1} \xrightarrow{\boldsymbol{x}'=\boldsymbol{W}^{\mathrm{T}}\boldsymbol{x}} \boldsymbol{x}'_{d\times 1}$。

需要特别说明的是，在步骤(3)中，$\boldsymbol{X}$ 的每一列均是中心化以后的数据。注意到协方差矩阵的系数$\frac{1}{m}$不会影响特征根的大小排序，也不会影响特征向量，所以这里计算 $\boldsymbol{C}$ 时忽略了协方差矩阵的系数$\frac{1}{m}$。为了叙述方便，这里将忽略了系数$\frac{1}{m}$的 $\boldsymbol{C}$ 仍然称为协方差矩阵。

注意到在特征向量上投影的方差恰好是该特征向量所属的特征根，在该方向上投影的方差在总方差中所占的比例便可以由该特征根在所有特征根中所占的比例描述。该比例通常称为贡献率，是选择有效主成分的重要依据。

在实际应用中，通常需要确定目标维度 d。由协方差矩阵 $\boldsymbol{C}$ 的特性可知，目标维度 d 不能超过样本个数，也不能超过原始数据的特征维度。具体取值可以利用经验，通过不断尝试的方式确定合适的目标维度。更为可靠的方式是使用累计贡献率确定目标维度 d，其定义为

$$\frac{\sum_{i=1}^{d}\lambda_i}{\sum_{i=1}^{n}\lambda_i}$$

其中，λ_1，λ_2，…，λ_n 是协方差矩阵 $\boldsymbol{C}$ 的特征根，且满足 $\lambda_1 \geqslant \lambda_2 \geqslant \cdots \geqslant \lambda_n$。在确定目标维度 d 时，预先设定好累计贡献率的阈值，如 95%，然后求使得累计贡献率超过 95%的最小的 d 值，将其作为目标维度。

上述算法的降维过程中 $\boldsymbol{x}'=\boldsymbol{W}^{\mathrm{T}}\boldsymbol{x}$，$\boldsymbol{x}$ 是训练集中经过中心化后的样本，那么对于测试样本 $\boldsymbol{x}_{\mathrm{test}}$ 应该如何降维？考虑到测试样本需要与训练样本同分布的要求，需要使用训练样本的均值对测试样本中心化，然后投影，即

$$\boldsymbol{x}'_{\mathrm{test}}=\boldsymbol{W}^{\mathrm{T}}(\boldsymbol{x}_{\mathrm{test}}-\boldsymbol{\mu})$$

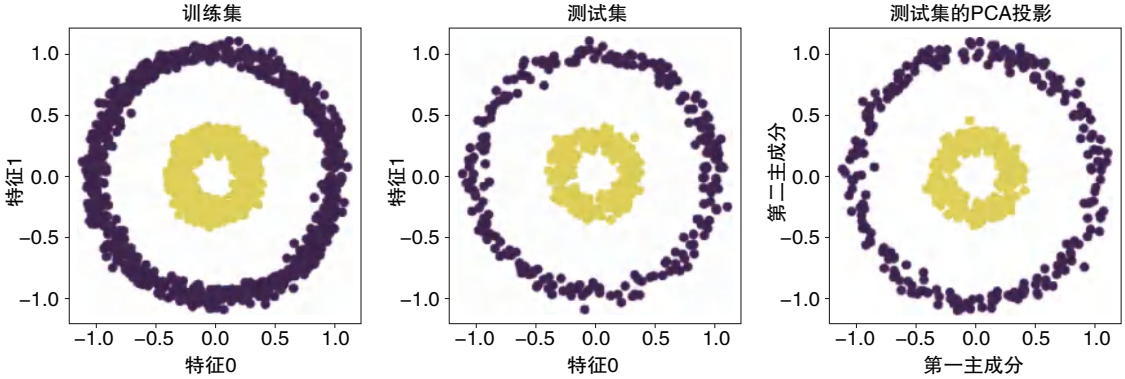
其中，$\boldsymbol{\mu}=\frac{1}{m}\sum_{i=1}^{m}\boldsymbol{x}_i$ 是训练样本的均值。

7.2.2 核主成分分析

由 7.2.1 节所述的投影过程可知，使用主成分分析降维是将样本与投影矩阵相乘，从而达到降维的目的，这是一种典型的线性投影，通常称为线性降维。该降维方法适用于协方差矩阵的特征根有较大差异的数据，如果数据的协方差矩阵的特征根差别不大，那么每个投影方向的贡献率将趋于相同，此时已经不适合再去挑选主成分，也就不宜使用主成分分析降维。

图 7-9 是说明主成分分析的局限性的一个例子。使用图 7-9(a)中训练样本所得投影矩阵，对图 7-9(b)中的测试样本进行投影，结果如图 7-9(c)所示，其中，协方差矩阵的特征根分别是 0.28 和 0.27。可以发现投影前后的数据分布基本没有变化，究其根源在于特征根非常接近，导致在第一主成分和第二主成分两个方向上的投影在分散程度上没有明显差异。此时，如果单纯考虑降维

问题，那么投影到第一主成分将很大程度地破坏原始数据的结构，并且损失将近 50%的信息。如果考虑分类问题，那么对当前的分类任务来说，主成分分析将没有任何作用。



（a）训练样本　（b）测试样本　（c）测试样本的PCA投影

图 7-9　主成分分析的局限性

针对以上线性降维存在的问题，类似于支持向量机中的核技巧，可以先将原始数据映射到高维空间，然后在高维特征空间中使用主成分分析进行降维，如图 7-10 所示。这种方法称为核主成分分析(Kernel-PCA)，是一种非线性降维技术。

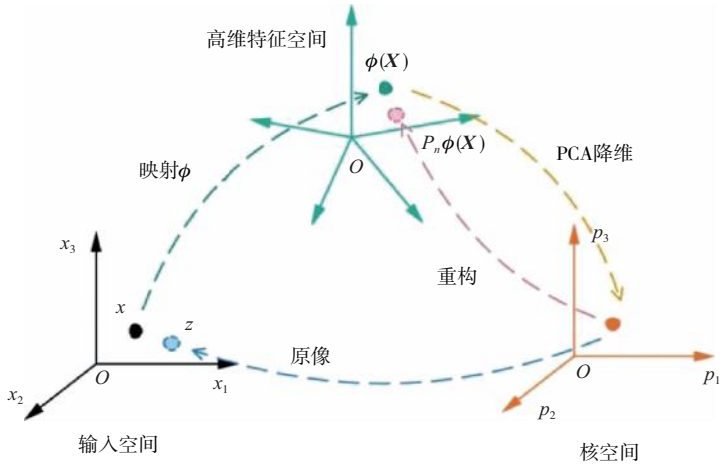


图 7-10　核主成分分析示意图

假设 ϕ 是从原始特征空间到高维特征空间的映射，类似于 $\boldsymbol{X}$ 的构造方式，在高维特征空间中将每个样本的像 $\phi(\boldsymbol{x}_i)$按列排成矩阵，记为 $\phi(\boldsymbol{X})$，即 $\phi(\boldsymbol{X})=[\phi(\boldsymbol{x}_1), \phi(\boldsymbol{x}_2), \cdots, \phi(\boldsymbol{x}_m)]$。这里假设在高维空间中 $\phi(\boldsymbol{x}_i)$已经中心化，具体的中心化问题将在后文中进一步讨论。

由 7.2.1 节的讨论可知，在高维特征空间中进行主成分分析只需对协方差矩阵进行特征根分解即可。为此，令 $\boldsymbol{C}=\phi(\boldsymbol{X})\phi(\boldsymbol{X})^{\mathrm{T}}$ 是高维空间中的协方差矩阵。与 7.2.1 节一样，在协方差矩阵中省略系数$\frac{1}{m}$并不影响主成分，所以这里也将其去掉。

考虑 $\boldsymbol{C}$ 的特征根分解问题$\boldsymbol{CV}=\lambda\boldsymbol{V}$，即

$$\phi(\boldsymbol{X})\phi(\boldsymbol{X})^{\mathrm{T}}\boldsymbol{V}=\lambda\boldsymbol{V} \tag{7-1}$$

这里只关注 $\lambda>0$ 的情形，因为 $\lambda=0$ 的特征向量已经不是主成分。将上式重写为 $\boldsymbol{V}=\phi(\boldsymbol{X})\frac{\phi(\boldsymbol{X})^{\mathrm{T}}\boldsymbol{V}}{\lambda}$，其中，$\frac{\phi(\boldsymbol{X})^{\mathrm{T}}\boldsymbol{V}}{\lambda}$显然为 m 维列向量，并将其记为 $\boldsymbol{\alpha}$，则

$$\boldsymbol{V}=\phi(\boldsymbol{X})\boldsymbol{\alpha} \tag{7-2}$$

可见高维空间中的投影方向是 $\phi(\boldsymbol{x}_1)$，$\phi(\boldsymbol{x}_2)$，…，$\phi(\boldsymbol{x}_m)$的线性组合，其系数向量为 $\boldsymbol{\alpha}$。在给定 ϕ 的情况下，只要能够求得 $\boldsymbol{\alpha}$，即可获得投影方向。为此，将式(7-2)代回式(7-1)中，可得

$$\phi(\boldsymbol{X})\phi(\boldsymbol{X})^{\mathrm{T}}\phi(\boldsymbol{X})\boldsymbol{\alpha}=\lambda\phi(\boldsymbol{X})\boldsymbol{\alpha} \tag{7-3}$$

两边同时左乘 $\phi(\boldsymbol{X})^{\mathrm{T}}$，并令 $\boldsymbol{K}=\phi(\boldsymbol{X})^{\mathrm{T}}\phi(\boldsymbol{X})$，则得到

$$\boldsymbol{K}^2\boldsymbol{\alpha}=\lambda\boldsymbol{K}\boldsymbol{\alpha} \tag{7-4}$$

如果 $\boldsymbol{K}$ 的特征根均大于零，则 $\boldsymbol{K}$ 可逆，那么式(7-4)等价于

$$\boldsymbol{K}\boldsymbol{\alpha}=\lambda\boldsymbol{\alpha} \tag{7-5}$$

如果 $\boldsymbol{K}$ 存在零特征根，那么零特征根对应的特征向量将导致“存在满足 $\boldsymbol{K}^2\boldsymbol{\alpha}=\lambda\boldsymbol{K}\boldsymbol{\alpha}$ 的 $\boldsymbol{\alpha}$ 不满足 $\boldsymbol{K}\boldsymbol{\alpha}=\lambda\boldsymbol{\alpha}$”。而 $\boldsymbol{K}$ 的零特征根对应的特征向量构建的下述投影矩阵为零矩阵 $\boldsymbol{V}=\phi(\boldsymbol{X})\boldsymbol{\alpha}$，所以，对于在高维空间中构建的投影方向 $\boldsymbol{V}$ 来说，$\boldsymbol{K}^2\boldsymbol{\alpha}=\lambda\boldsymbol{K}\boldsymbol{\alpha}$ 和 $\boldsymbol{K}\boldsymbol{\alpha}=\lambda\boldsymbol{\alpha}$ 的解的差异性无关紧要。因此，直接求解 $\boldsymbol{K}\boldsymbol{\alpha}=\lambda\boldsymbol{\alpha}$ 即可。

上述矩阵 $\boldsymbol{K}$ 称为核矩阵，可以证明核矩阵是半正定的。由上述核矩阵 $\boldsymbol{K}$ 的定义可知，需要确定从低维空间到高维空间的映射 ϕ。但是这个映射往往难以确定，而且在高维空间直接进行运算也会带来非常大的计算开销。与 6.1.2 节一样，一个行之有效的变通方法称为核技巧，即寻找一个核函数 $\kappa(\cdot,\cdot)$，由该函数定义核矩阵 $\boldsymbol{K}$：i 行 j 列元素为 $k_{ij}=\kappa(\boldsymbol{x}_i,\boldsymbol{x}_j)$。

有了核矩阵便无须显式地给出低维空间到高维空间的映射，而且避免了在高维空间进行运算量较大的内积运算。常用的核函数有多项式核(polynomial kernel)函数、线性核(linear kernel)函数、径向基核(radial basis kernel)函数及 Sigmoid 核函数，表达式如表 7-3 所示。特别地，当核函数取为线性核函数时，核主成分分析退化为主成分分析。

表 7-3　常用核函数及其表达式

核函数	表达式
线性核函数	$\kappa(\boldsymbol{x}_i,\boldsymbol{x}_j)=\boldsymbol{x}_i^{\mathrm{T}}\boldsymbol{x}_j$
多项式核函数	$\kappa(\boldsymbol{x}_i,\boldsymbol{x}_j)=(\gamma\boldsymbol{x}_i^{\mathrm{T}}\boldsymbol{x}_j+r)^d$
径向基核函数	$\kappa(\boldsymbol{x}_i,\boldsymbol{x}_j)=\mathrm{e}^{-\gamma\lVert\boldsymbol{x}_i-\boldsymbol{x}_j\rVert^2}$，$\gamma>0$
Sigmoid 核函数	$\kappa(\boldsymbol{x}_i,\boldsymbol{x}_j)=\tanh(\gamma\boldsymbol{x}_i^{\mathrm{T}}\boldsymbol{x}_j+r)$

通过对核矩阵 $\boldsymbol{K}$ 进行特征根分解，可以求出最大特征根对应的特征向量 $\boldsymbol{\alpha}$，接下来解决投影方向 $\boldsymbol{V}=\phi(\boldsymbol{X})\boldsymbol{\alpha}$ 的单位化问题。如 7.2.1 节所述，这里要求投影方向 $\boldsymbol{V}$ 是单位向量，即要求 $\boldsymbol{V}^{\mathrm{T}}\boldsymbol{V}=\boldsymbol{\alpha}^{\mathrm{T}}\phi(\boldsymbol{X})^{\mathrm{T}}\phi(\boldsymbol{X})\boldsymbol{\alpha}=\boldsymbol{\alpha}^{\mathrm{T}}\boldsymbol{K}\boldsymbol{\alpha}=\lambda\boldsymbol{\alpha}^{\mathrm{T}}\boldsymbol{\alpha}=1$，为此令 $\boldsymbol{\alpha}'=\frac{1}{\sqrt{\lambda}\lVert\boldsymbol{\alpha}\rVert}\boldsymbol{\alpha}$，则 $V=\phi(\boldsymbol{X})\boldsymbol{\alpha}'$即为单位向量。后文为了叙述方便，将 $\boldsymbol{\alpha}'$直接记为 $\boldsymbol{\alpha}$。

在已经获得投影方向的情况下，接下来考虑如何投影。因为有

$$\boldsymbol{x}_i'=\boldsymbol{V}^{\mathrm{T}}\phi(\boldsymbol{x}_i)=\boldsymbol{\alpha}^{\mathrm{T}}\phi(\boldsymbol{X})^{\mathrm{T}}\phi(\boldsymbol{x}_i)$$

其中，$\phi(\boldsymbol{X})^{\mathrm{T}}\phi(\boldsymbol{x}_i)$恰好是核矩阵 $\boldsymbol{K}$ 的第 i 列，记为 $\boldsymbol{K}(:,\ i)$。所以，在高维空间中 $\phi(\boldsymbol{x}_i)$ 在 $\boldsymbol{V}$ 方向上的投影为

$$\boldsymbol{x}_i'=\boldsymbol{\alpha}^{\mathrm{T}}\boldsymbol{K}(:,\ i)$$

上述过程假设在高维空间中，$\phi(\boldsymbol{x}_i)(i=1,\ 2,\ \cdots,\ m)$均已中心化。接下来考虑中心化问题。

先考虑在高维空间中，中心化 $\phi(\boldsymbol{x}_i)(i=1,\ 2,\ \cdots,\ m)$以后核矩阵的表示形式。为此，令 $\mathbf{1}_{m\times1}=(1)_{m\times1}$ 表示每个元素均为 1 的 m 维列向量，则

$$\tilde{\phi}(\boldsymbol{x}_i)=\phi(\boldsymbol{x}_i)-\frac{1}{m}\sum_{j=1}^{m}\phi(\boldsymbol{x}_j)=\phi(\boldsymbol{x}_i)-\frac{1}{m}\phi(\boldsymbol{X})\mathbf{1}_{m\times1}$$

令 $\mathbf{1}_{1\times m}=(1)_{1\times m}$ 表示每个元素均为 1 的 m 维行向量，且 $\mathbf{1}_m=\left(\frac{1}{m}\right)_{m\times m}$ 表示每个元素均为$\frac{1}{m}$的 m 阶矩阵。则

$$\begin{aligned}\tilde{\phi}(\boldsymbol{X})&=\left[\phi(\boldsymbol{x}_1)-\frac{1}{m}\phi(\boldsymbol{X})\mathbf{1}_{m\times1},\ \phi(\boldsymbol{x}_2)-\frac{1}{m}\phi(\boldsymbol{X})\mathbf{1}_{m\times1},\ \cdots,\ \phi(\boldsymbol{x}_m)-\frac{1}{m}\phi(\boldsymbol{X})\mathbf{1}_{m\times1}\right]\\&=[\phi(\boldsymbol{x}_1),\ \phi(\boldsymbol{x}_2),\ \cdots,\ \phi(\boldsymbol{x}_m)]-\frac{1}{m}\phi(\boldsymbol{X})\mathbf{1}_{m\times1}\mathbf{1}_{1\times m}\\&=\phi(\boldsymbol{X})-\frac{1}{m}\phi(\boldsymbol{X})\mathbf{1}_{m\times1}\mathbf{1}_{1\times m}\\&=\phi(\boldsymbol{X})-\phi(\boldsymbol{X})\mathbf{1}_m\end{aligned}$$

所以中心化后的核矩阵为

$$\begin{aligned}\tilde{\boldsymbol{K}}&=\tilde{\phi}(\boldsymbol{X})^{\mathrm{T}}\tilde{\phi}(\boldsymbol{X})\\&=[\phi(\boldsymbol{X})^{\mathrm{T}}-\mathbf{1}_m\phi(\boldsymbol{X})^{\mathrm{T}}][\phi(\boldsymbol{X})-\phi(\boldsymbol{X})\mathbf{1}_m]\\&=\phi(\boldsymbol{X})^{\mathrm{T}}\phi(\boldsymbol{X})-\phi(\boldsymbol{X})^{\mathrm{T}}\phi(\boldsymbol{X})\mathbf{1}_m-\mathbf{1}_m\phi(\boldsymbol{X})^{\mathrm{T}}\phi(\boldsymbol{X})+\mathbf{1}_m\phi(\boldsymbol{X})^{\mathrm{T}}\phi(X)\mathbf{1}_m\\&=\boldsymbol{K}-\boldsymbol{K}\mathbf{1}_m-\mathbf{1}_m\boldsymbol{K}+\mathbf{1}_m\boldsymbol{K}\mathbf{1}_m\end{aligned}$$

与 PCA 一样，对于测试样本，也需要中心化处理后再投影。接下来考虑关于测试样本 $\boldsymbol{x}_{\text{test}}$ 的中心化及投影。中心化时使用训练样本在高维空间中的均值，即

$$\tilde{\phi}(\boldsymbol{x}_{\text{test}})=\phi(\boldsymbol{x}_{\text{test}})-\frac{1}{m}\phi(\boldsymbol{X})\mathbf{1}_{m\times1}$$

然后在方向 $\boldsymbol{V}=\tilde{\phi}(\boldsymbol{X})\boldsymbol{\alpha}$ 上投影，结果为

$$\begin{aligned}\boldsymbol{x}_{\text{test}}'&=\boldsymbol{V}^{\mathrm{T}}\tilde{\phi}(\boldsymbol{x}_{\text{test}})=\boldsymbol{\alpha}^{\mathrm{T}}\tilde{\phi}(\boldsymbol{X})^{\mathrm{T}}\tilde{\phi}(\boldsymbol{x}_{\text{test}})\\&=\boldsymbol{\alpha}^{\mathrm{T}}[\phi(\boldsymbol{X})^{\mathrm{T}}-\mathbf{1}_m\phi(\boldsymbol{X})^{\mathrm{T}}]\left[\phi(\boldsymbol{x}_{\text{test}})-\frac{1}{m}\phi(\boldsymbol{X})\mathbf{1}_{m\times1}\right]\\&=\boldsymbol{\alpha}^{\mathrm{T}}\left(\boldsymbol{K}_{\text{test}}-\frac{1}{m}\boldsymbol{K}\mathbf{1}_{m\times1}-\mathbf{1}_m\boldsymbol{K}_{\text{test}}+\frac{1}{m}\mathbf{1}_m\boldsymbol{K}\mathbf{1}_{m\times1}\right)\\&=\boldsymbol{\alpha}^{\mathrm{T}}\tilde{\boldsymbol{K}}_{\text{test}}\end{aligned}$$

其中，$\tilde{\boldsymbol{K}}_{\text{test}}=\boldsymbol{K}_{\text{test}}-\mathbf{1}_m\boldsymbol{K}_{\text{test}}-\frac{1}{m}\boldsymbol{K}\mathbf{1}_{m\times1}+\frac{1}{m}\mathbf{1}_m\boldsymbol{K}\mathbf{1}_{m\times1}$，且 $\boldsymbol{K}_{\text{test}}=\phi(\boldsymbol{X})^{\mathrm{T}}\phi(\boldsymbol{x}_{\text{test}})$是 m 维列向量，

由核函数 $\kappa(\cdot,\cdot)$ 确定：第 i 个元素为 $k_i=\kappa(\boldsymbol{x}_i,\boldsymbol{x}_{\text{test}})$。

综上所述，核主成分分析的训练阶段主要是确定投影矩阵的系数，在测试阶段主要通过上述系数计算投影结果。训练过程描述如下。

输入：样本集 $D=\{\boldsymbol{x}_1,\boldsymbol{x}_2,\cdots,\boldsymbol{x}_m\}$；降维后的维度 d。

步骤：

(1)计算核矩阵 $\boldsymbol{K}=(k_{ij})$，$k_{ij}=\kappa(\boldsymbol{x}_i,\boldsymbol{x}_j)$；

(2)中心化 $\widetilde{\boldsymbol{K}}=\boldsymbol{K}-\boldsymbol{K}\boldsymbol{1}_m-\boldsymbol{1}_m\boldsymbol{K}+\boldsymbol{1}_m\boldsymbol{K}\boldsymbol{1}_m$，其中，$\boldsymbol{1}_m=\left(\frac{1}{m}\right)_{m\times m}$；

(3)特征根分解并单位化 $\widetilde{\boldsymbol{K}}\boldsymbol{\alpha}=\lambda\boldsymbol{\alpha}$，$\boldsymbol{\alpha}\to\frac{1}{\|\boldsymbol{\alpha}\|\sqrt{\lambda}}\boldsymbol{\alpha}$；

(4)确定投影矩阵的系数，取最大的 d 个特征根对应的特征向量 $\boldsymbol{\alpha}_1,\boldsymbol{\alpha}_2,\cdots,\boldsymbol{\alpha}_d$。

输出：降维的系数矩阵 $\boldsymbol{A}=[\boldsymbol{\alpha}_1,\boldsymbol{\alpha}_2,\cdots,\boldsymbol{\alpha}_d]_{m\times d}$。

在训练完成后可以得到核矩阵 $\boldsymbol{K}$ 和系数矩阵 $\boldsymbol{A}=[\boldsymbol{\alpha}_1,\boldsymbol{\alpha}_2,\cdots,\boldsymbol{\alpha}_d]_{m\times d}$，之后便可以对测试样本 $\boldsymbol{x}_{\text{test}}$ 进行降维，降维过程描述如下。

输入：测试样本 $\boldsymbol{x}_{\text{test}}$；降维后的维度 d。

步骤：

(1)计算测试样本的核矩阵 $\boldsymbol{K}_{\text{test}}=(k_i)$，$k_i=\kappa(\boldsymbol{x}_i,\boldsymbol{x}_{\text{test}})$；

(2)中心化 $\widetilde{\boldsymbol{K}}_{\text{test}}=\boldsymbol{K}_{\text{test}}-\boldsymbol{1}_m\boldsymbol{K}_{\text{test}}-\frac{1}{m}\boldsymbol{K}\boldsymbol{1}_{m\times 1}+\frac{1}{m}\boldsymbol{1}_m\boldsymbol{K}\boldsymbol{1}_{m\times 1}$，其中，$\boldsymbol{1}_m=\left(\frac{1}{m}\right)_{m\times m}$，$\boldsymbol{1}_{m\times 1}=(1)_{m\times 1}$；

(3)降维 $\boldsymbol{x}'_{\text{test}}=\boldsymbol{A}^{\mathrm{T}}\widetilde{\boldsymbol{K}}_{\text{test}}$。

输出：降维后的样本 $\boldsymbol{x}'_{\text{test}\,d\times 1}$。

如果给定一批测试样本 $\boldsymbol{x}^1_{\text{test}},\boldsymbol{x}^2_{\text{test}},\cdots,\boldsymbol{x}^l_{\text{test}}$，可以一次性进行降维。此时，测试样本的核矩阵 $\boldsymbol{K}_{\text{test}}$ 为 $k_{ij}=\kappa(\boldsymbol{x}_i,\boldsymbol{x}^j_{\text{test}})$。中心化结果为

$$\widetilde{\boldsymbol{K}}_{\text{test}}=\boldsymbol{K}_{\text{test}}-\boldsymbol{1}_m\boldsymbol{K}_{\text{test}}-\frac{1}{m}\boldsymbol{K}\boldsymbol{1}_{m\times l}+\frac{1}{m}\boldsymbol{1}_m\boldsymbol{K}\boldsymbol{1}_{m\times l}$$

其中，$\boldsymbol{1}_{m\times l}$ 是 m 行 l 列全 1 矩阵。使用 $\boldsymbol{X}'_{\text{test}}=\boldsymbol{A}^{\mathrm{T}}\widetilde{\boldsymbol{K}}_{\text{test}}$ 得到测试样本 $\boldsymbol{x}^1_{\text{test}},\boldsymbol{x}^2_{\text{test}},\cdots,\boldsymbol{x}^l_{\text{test}}$ 的降维结果，其中，$\boldsymbol{X}'_{\text{test}}$ 是 d 行 l 列矩阵，每一列对应一个测试样本。

7.3 其他算法

7.3.1 谱聚类算法

如果将每个待聚类的样本看作一个顶点，样本之间的联系通过顶点之间边的权重来描述，连接同一类顶点的边的权重很大，而不同类顶点之间边的权重很小，就可以得到一幅有权图。再进一步，如果能够找到一种图的切割方法，沿着权重较小的边对图进行切割，使得切割之后各个子图内的权重均很大，那么切割出来的子图就是聚类结果。这正是谱聚

类算法的核心思想，该算法包括构图和切图两个核心步骤。

在构图方面，一般有 3 种常见方式：ε 邻域法、k 近邻法和全连接法。前两种方法构造出的图的邻接矩阵往往是稀疏矩阵，比较适合样本量大的聚类任务。

谱聚类切图目前有两种主流方式：RatioCut 和 NCut，其目的是找到一组权重最小，又能平衡切出子图的边。这里给出基于 NCut 切图的算法描述。

输入：$D=\{\boldsymbol{x}_1, \boldsymbol{x}_2, \cdots, \boldsymbol{x}_m\}$，簇数 k。

步骤：

(1)选择一种构图方式，构造邻接矩阵 $\boldsymbol{W}$；

(2)计算度矩阵 $\boldsymbol{D}$，其为对角阵，对角线元素是对应顶点与其他顶点之间边的权重之和，即邻接矩阵 $\boldsymbol{W}$ 中对应行(或者对应列)的和。

(3)计算图拉普拉斯矩阵 $\boldsymbol{L}=\boldsymbol{D}-\boldsymbol{W}$；

(4)对 $\boldsymbol{L}$ 标准化 $\boldsymbol{L}=\boldsymbol{D}^{-\frac{1}{2}}\boldsymbol{L}\boldsymbol{D}^{-\frac{1}{2}}$；

(5)对 $\boldsymbol{L}$ 进行特征值分解，将其最小的 k 个特征值对应的特征向量(列向量)排成矩阵(m 行 k 列)$\boldsymbol{Q}$；

(6)选择常规聚类算法，如 k 均值聚类算法，对 $\boldsymbol{Q}$ 的各行进行聚类，得到对 $\boldsymbol{Q}$ 的聚类结果$\{A_1, A_2, \cdots, A_k\}$；

(7)将$\{A_1, A_2, \cdots, A_k\}$对应到原始数据的聚类结果$\{C_1, C_2, \cdots, C_k\}$上，如果 $\boldsymbol{Q}$ 中第 i 行属于簇 A_j，那么第 i 个顶点对应的样本属于簇 C_j。

输出：$\{C_1, C_2, \cdots, C_k\}$。

谱聚类算法在非凸数据方面有明显优势，而且对稀疏数据和高维度数据均有比较理想的聚类结果。图 7-11 展示了谱聚类算法在 4 种不同数据集上的聚类结果，可以看出该算法对不同分布及不同结构的数据均能很好地进行聚类。

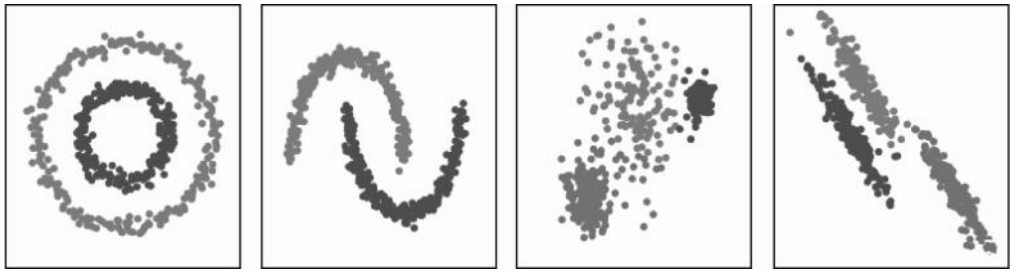


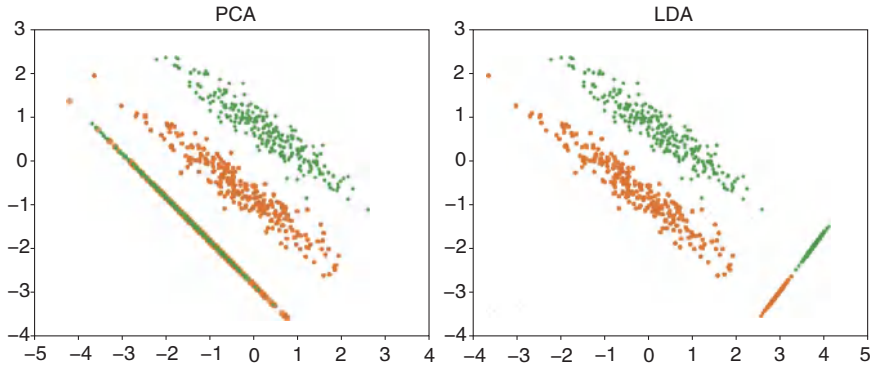
图 7-11　谱聚类算法在不同数据集上的聚类结果

7.3.2　LDA 算法

回顾 7.2.1 节，使用 PCA 降维时并未使用类别信息，单纯从数据分布入手，将其投影到方差尽可能大的方向上。但是在诸如分类等任务中，可能会出现投影后对分类无益的情况，甚至有时会出现非常糟糕的现象：原本线性可分的数据投影后不再线性可分，如图 7-12(a)所示。出现这一问题的原因就在于 PCA 是无监督学习方法，降维过程中并未有效利用已有的类别信息。

线性判别分析(linear discriminant analysis，LDA)算法很好地回避了这个问题，该算法

有效地利用了样本的类别信息，在确定投影方向的时候同时考虑投影后类内距离最小且类间距离最大。也就是期望同一种类别的数据在投影后尽可能接近，而不同类别的数据在投影后的类别中心之间的距离尽可能大。在此原则下，如图 7-12(b)所示，使用 LDA 投影后，所得数据仍然保持了原始数据线性可分的特性，这在分类任务中尤为重要。



（a）使用PCA投影　　（b）使用LDA投影

图 7-12　在部分分类任务中 LDA 算法相比 PCA 算法的优势

显然，与 PCA 算法不同的是，LDA 算法是监督学习方法。考虑到 LDA 算法与 PCA 算法一样同属降维算法，所以这里一并介绍 LDA 算法。

投影后类内距离最小且类间距离最大的思想是通过类内散度矩阵和类间散度矩阵来实现的。设样本集为 $D=\{\boldsymbol{x}_1, \boldsymbol{x}_2, \cdots, \boldsymbol{x}_m\}$，样本 $\boldsymbol{x}_i$ 的标注信息为 y_i，类别集合为 $C=\{c_i \mid c_i$ 是类别标记，$i\leqslant k$，k 是类别数$\}$，第 i 类样本的集合记为 $X_i=\{\boldsymbol{x}_j \mid y_j=c_i\}$。以 m_i 表示第 i 类样本的个数，即 $m_i=|X_i|$，以 $\boldsymbol{\mu}_i$ 表示第 i 类样本的均值，即 $\boldsymbol{\mu}_i=\frac{1}{m_i}\sum_{\boldsymbol{x}\in X_i}\boldsymbol{x}$。在给定目标维度 d 的情况下，LDA 算法的具体降维步骤如下：

(1)计算类内散度矩阵

$$\boldsymbol{S}_w=\sum_i\sum_{\boldsymbol{x}\in X_i}(\boldsymbol{x}-\boldsymbol{\mu}_i)(\boldsymbol{x}-\boldsymbol{\mu}_i)^{\mathrm{T}}$$

(2)计算类间散度矩阵

$$\boldsymbol{S}_b=\sum_i m_i(\boldsymbol{\mu}_i-\boldsymbol{\mu})(\boldsymbol{\mu}_i-\boldsymbol{\mu})^{\mathrm{T}}$$

(3)广义特征值分解

$$\boldsymbol{S}_b\boldsymbol{w}=\lambda\boldsymbol{S}_w\boldsymbol{w}$$

并取最大的 d 个广义特征值对应的特征向量(列向量)，构造投影矩阵

$$\boldsymbol{W}=(\boldsymbol{w}_1, \boldsymbol{w}_2, \cdots, \boldsymbol{w}_d)$$

(4)投影

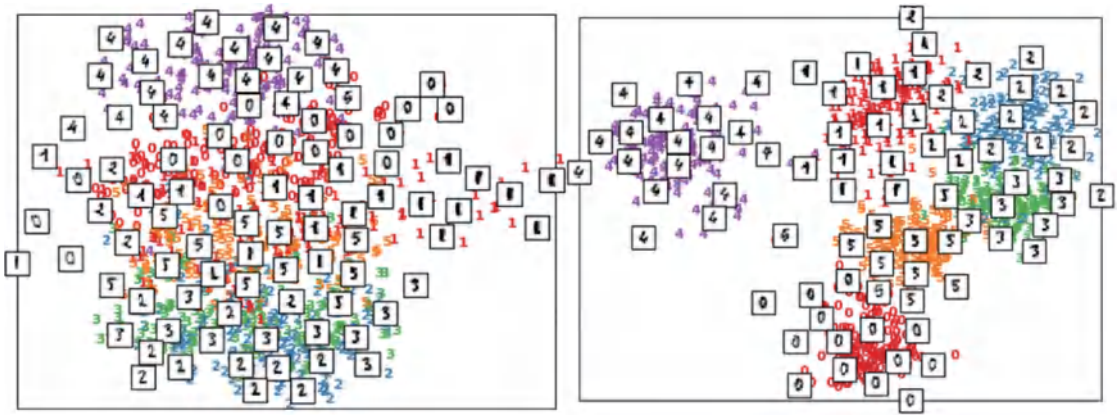
$$\boldsymbol{z}_i=\boldsymbol{W}^{\mathrm{T}}\boldsymbol{x}_i$$

输出：

$$(\boldsymbol{z}_1, y_1), (\boldsymbol{z}_2, y_2), \cdots, (\boldsymbol{z}_m, y_m)$$

此外，关于目标维度 d 的确定可以参考 PCA 算法的方式。需要特别注意，这里的目标维度不能大于原始特征维度，同时也不能超过 $k-1$。

最后通过一个实例进一步比较 PCA 和 LDA 算法在分类任务中的表现。针对手写体数字数据集，选取其类别为 0～5 的 6 类样本，对其特征(1 024 个维度)进行降维。为了可视化方便，将目标维度设置为 2。分别使用 PCA 和 LDA 算法进行降维，降维后的可视化结果如图 7-13 所示。比较图 7-13 中(a)和(b)的结果，可以发现相比于 PCA 算法，LDA 算法的结果中同一类的样本更加集中，而不同类的样本之间更加分散。特别是数字 0 的样本，PCA 算法的结果中这些样本分布得很分散，与数字 1、2、4 和 5 的样本均相邻，而 LDA 算法的结果中，数字 0 的样本很集中，而且只与数字 5 的样本相邻，显然这种特性更有助于完成手写体数字识别任务。



(a) PCA　　(b) LDA

图 7-13　PCA 和 LDA 算法在手写体数字数据集上降维效果的比较

习　　题

1. k 均值++算法是聚类算法吗？简述它和 k 均值算法的关系。
2. 简述 DBSCAN 算法中参数 ε 和 MinPts 对聚类结果的影响。
3. 简述自底向上和自顶向下的层次聚类思想的异同。
4. 简述兰德指数在评价聚类模型时存在的问题。
5. 简述降维的目的及应用场景。

习题解析